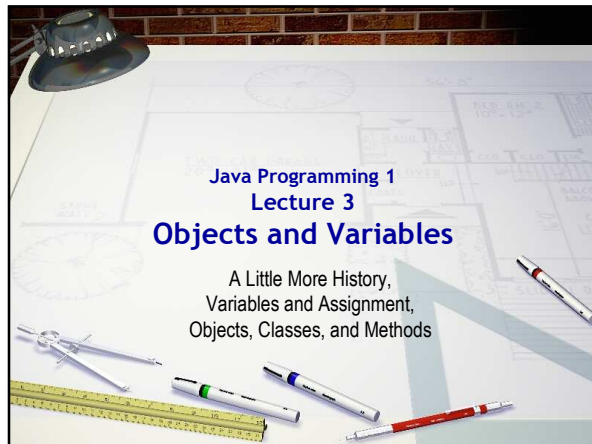
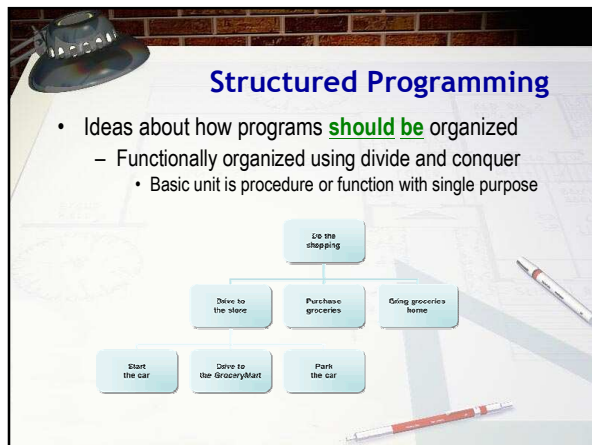






GOTO Considered Harmful

- Function should have single entrance, single exit
 - GOTO produced problem known as **spaghetti code**
 - Eliminated by application of sequence, selection, iteration
 - Boehm and Jacopini, Edsger Dijkstra

```
100 GOTO 500
110 PRINT I:
120 GOTO 400
130 PRINT I * 12:
140 GOTO 450
150 PRINT " = ";
160 GOTO 150
170 GOTO 150
180 PRINT " 12 ";
190 GOTO 200
200 PRINT " * ";
210 GOTO 300
220 T = I * 1
230 IF I > 12 THEN STOP
240 GOTO 110
250 PRINT "The Twelves Table"
260 T = 1
270 GOTO 110
```

Information Hiding

- With thousands of procedures, programs accidentally modified data used by another procedure
- Solution: "hide" data inside procedure: **local variables**
- Required a new type of language, called **block-structured** languages
- Pascal and C

```

int main()
{
    firstStep();
    secondStep();
}

void firstStep()
{
    int a = 10;
}

void secondStep()
{
    int b = 3;
}

```

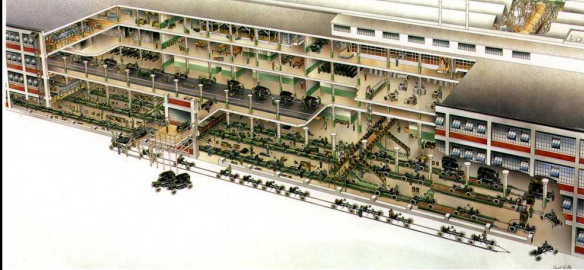
The main() function cannot see or change the variables a or b

The variable a is visible only inside the procedure firstStep(). It can be used and changed here but is invisible in all others.

The variable b can only be used inside of secondStep().

Structured Programming

- Organized around a hierarchy of procedures
 - Effective for linear, assembly-line type problems



Object-Oriented Programming

- Better for building "reactive" software like GUIs
 - "Communities" with **objects** as basic building block
 - Self-contained components that work together
 - Windows, buttons, menus, scrollbars, etc.



An O-O Vocabulary

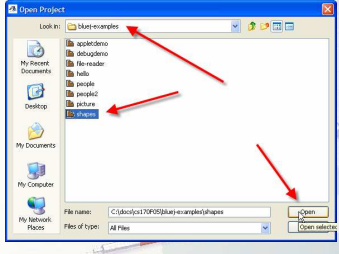
- Let's take a look at some O-O vocabulary terms
 - Objects**: "Variables" that contain both data, and the functions that operate on that data.
 - Classes**: "Blueprints" used to describe objects.
 - Encapsulation**: Hiding data inside an object
 - Inheritance**: Creating new classes from old
 - Polymorphism**: Different objects, same message
- Let's take a look at these using the BlueJ IDE

Express Yourself

- Use OpenOffice Writer to create a new document
- Save the file as LastFirst_ic03
 - Replace LastFirst with your actual name
- Put your name and today's date at the top of the sheet
- Title it "CS 170 In-class Exercise 3"
- Exercise 1**: Locate the examples folder in C:\Apps\BlueJ and copy the entire folder to your CS 170 Home folder on the U: drive. Snap a picture of the folder in Windows Explorer

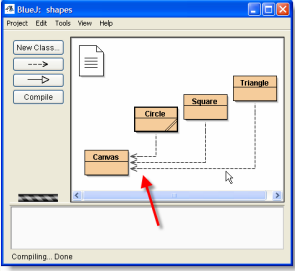
Examples

- Rename the folder to **bluej-examples**
 - So any changes you make will be there next time
- Choose **Open Project...** from the File menu
- Exercise 2**: Find and open the **Shapes** project and snap screen



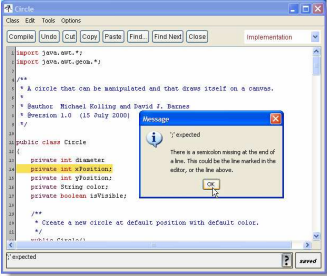
The Project Window

- Rectangle for each class
 - Diagonal lines mean “not compiled”
- Click **Compile** button
- Arrows represent class relationships
 - **Circle**, **Square**, **Triangle** **use** the **Canvas** for display
- Open **editor** by double-clicking the **Circle** class



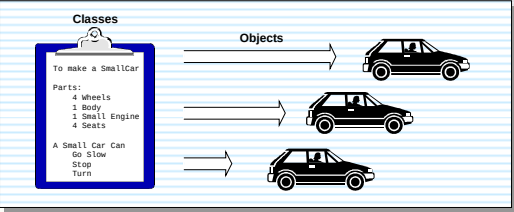
The Source Code

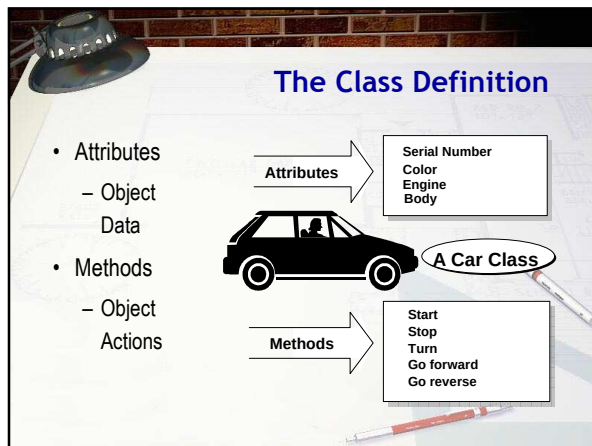
- Each source file gets an independent editor window
 - Remove semicolon after diameter and compile file
- **Exercise 3:** snap
 - Error highlighted, simple message
 - Extended help
- Editing not “full-featured” as SciTE
- What are classes?



What Are Classes?

- A **pattern** or **blueprint** that describes the common characteristics of a category of objects





Classes & Object Creation

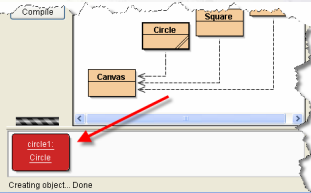
- When you **define a class**, you specify
 - The data attributes that hold its object's state
 - The methods that define object behavior
- When you **create an object**
 - You create an **instance** of a class
 - These instances are called objects
 - This is also called **instantiation**
- You **send messages** to objects
 - Each message **invokes** a particular method

What are Objects?

- **Components** used to build OO computer programs
 - User-defined variables used in OO programming
 - Represent visible objects, concepts, relationships
- All objects have three properties
 - **Identity**: who the object is
 - **State**: the object's characteristics
 - **Behavior**: what the object can do
- We'll use BlueJ to examine each of these properties

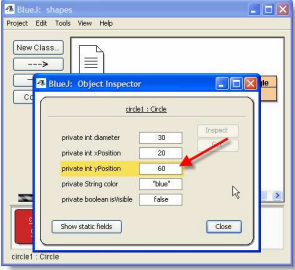
Creating Objects

- BlueJ allows you to **interactively** create objects and then send messages to those objects
- Let's create a few **Circle** objects
 - Right-click **Circle** class
 - Choose **new Circle()**
 - Name the **new** instance **circle1**
 - Object's **identity**
 - Create a 2nd **Circle**
- Exercise 4:** snap



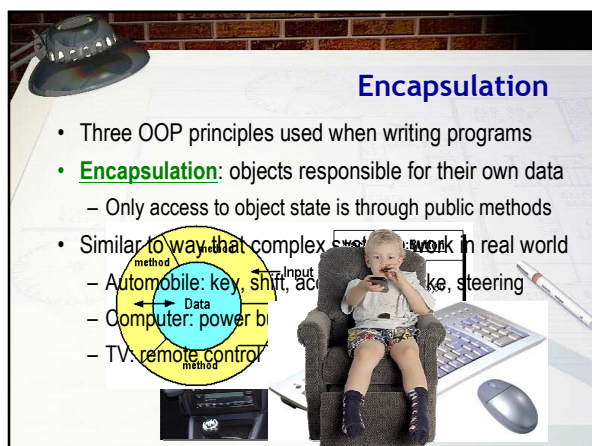
Object State

- State:** **values** stored in an object's **data fields**
 - In BlueJ, we can inspect the object to examine its state
 - Right-click the **Circle** object named **circle1** on the Object Bench
 - Choose **Inspect**
- Exercise 5:** snap
- Examine object's data
- Try changing one



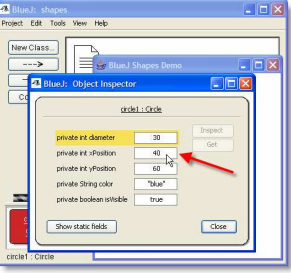
Encapsulation

- Three OOP principles used when writing programs
- Encapsulation:** objects responsible for their own data
 - Only access to object state is through public methods
- Similar to way that complex systems work in real world
 - Automobile: key, shift, accelerator, brake, steering
 - Computer: power button, mouse, keyboard
 - TV: remote control



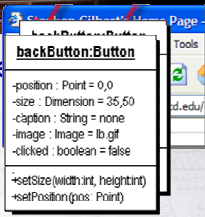
Object Behavior

- **Behavior**: things the object can do (**methods**)
 - Right-click **circle1** and send the **makeVisible()** message to the object
- Choose **moveRight()** and then **Inspect**
 - Note how state can change in response to a message
- All **Circle** methods are **mutator** methods



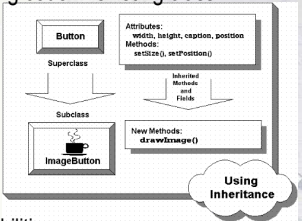
Object Properties

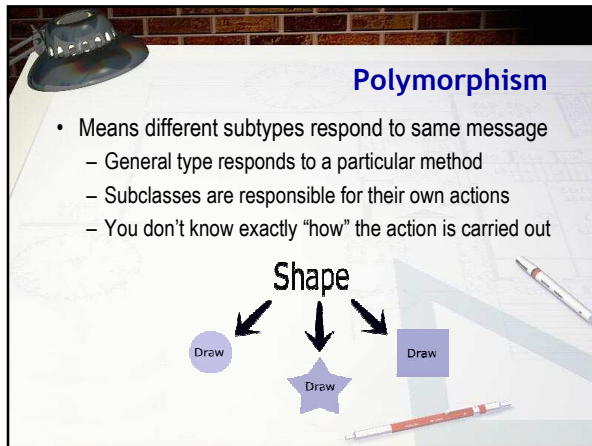
- **Identity**: the object's "name"
 - Your program can have several **Button** objects
 - Give each one a different name
- **State**: **values** stored in an object's **data fields**
 - One **Button** object may be active, another disabled
 - One may use a left arrow, another, a right arrow
- **Behavior**: things the object can do (**methods**)
 - A spreadsheet cell may "know" how to recalculate
 - A **Button** object can "be clicked"



Inheritance

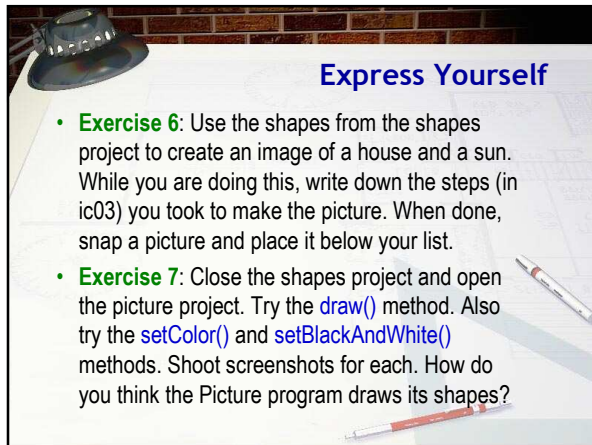
- Create a new **class** using code in existing class
 - Newly created class is called a **subclass**
 - Think of it as a "child" class
 - The "parent" class is called the **superclass**
 - Subclass usually has new attributes or capabilities
 - "Inherits" attributes and capabilities of its parent





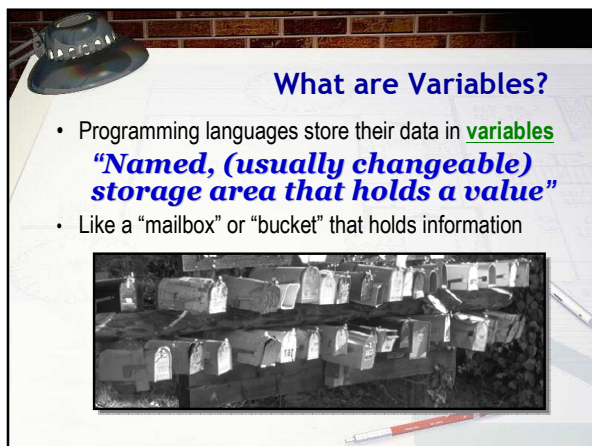
Polymorphism

- Means different subtypes respond to same message
 - General type responds to a particular method
 - Subclasses are responsible for their own actions
 - You don't know exactly "how" the action is carried out



Express Yourself

- Exercise 6:** Use the shapes from the shapes project to create an image of a house and a sun. While you are doing this, write down the steps (in ic03) you took to make the picture. When done, snap a picture and place it below your list.
- Exercise 7:** Close the shapes project and open the picture project. Try the `draw()` method. Also try the `setColor()` and `setBlackAndWhite()` methods. Shoot screenshots for each. How do you think the Picture program draws its shapes?



What are Variables?

- Programming languages store their data in **variables**
"Named, (usually changeable) storage area that holds a value"
- Like a "mailbox" or "bucket" that holds information



What are Values?

- The data stored inside a variable is called its **value**
 - A variable can have different values as a program runs
- Different **kinds** of values we can put in a variable
 - We can store different numbers **or** text in a variable

```
PI = 3.14159;  
myAge = 58;  
myStreetNumber = "575";
```

- **PI** holds a **real** number, **myAge** a **whole** number
- **myStreetNumber** contains characters
 - Can't perform arithmetic on **myStreetNumber**

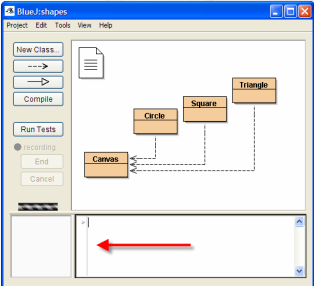
What are data types?

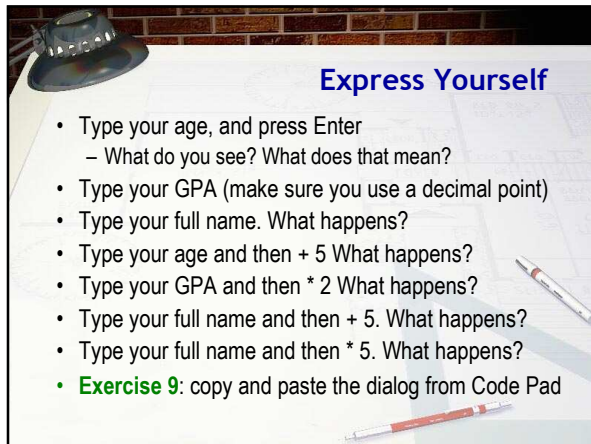
- Different **kinds** of variables for different **kinds** of data
- Different **sizes** of containers for **same kind** of data



Examining Types

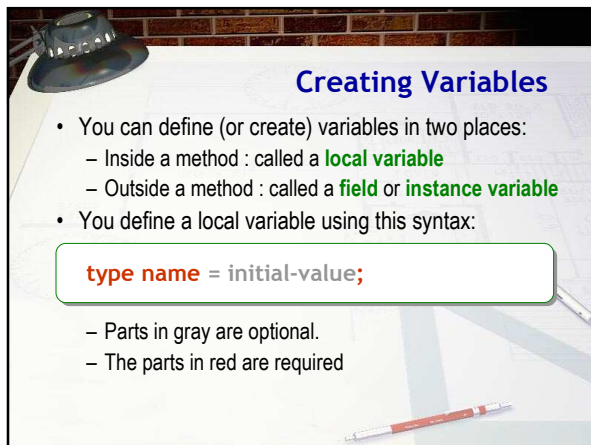
- Use Code Pad to examine types and variables
- Choose **Show Code Pad** from View menu
- Expand the Code Pad area like this
- **Exercise 8:** snap a picture





Express Yourself

- Type your age, and press Enter
 - What do you see? What does that mean?
- Type your GPA (make sure you use a decimal point)
- Type your full name. What happens?
- Type your age and then + 5 What happens?
- Type your GPA and then * 2 What happens?
- Type your full name and then + 5. What happens?
- Type your full name and then * 5. What happens?
- **Exercise 9:** copy and paste the dialog from Code Pad

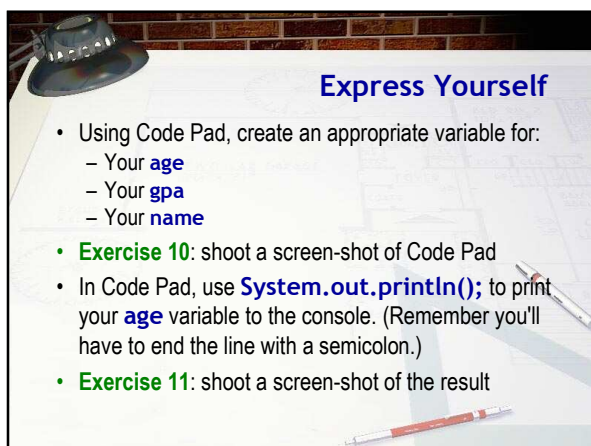


Creating Variables

- You can define (or create) variables in two places:
 - Inside a method : called a **local variable**
 - Outside a method : called a **field** or **instance variable**
- You define a local variable using this syntax:

```
type name = initial-value;
```

 - Parts in gray are optional.
 - The parts in red are required



Express Yourself

- Using Code Pad, create an appropriate variable for:
 - Your **age**
 - Your **gpa**
 - Your **name**
- **Exercise 10:** shoot a screen-shot of Code Pad
- In Code Pad, use **System.out.println();** to print your **age** variable to the console. (Remember you'll have to end the line with a semicolon.)
- **Exercise 11:** shoot a screen-shot of the result

Defining Local Variables

- So far, we've only learned about a few types
 - Type **int** stands for whole numbers
 - Type **double** stands for real numbers
 - Type **String** stands for a sequence of characters
- Here are some example declarations:

```
int x;  
double temperature;  
String college;
```

Placing Values in Variables

- Values are placed in variables using **assignment**

```
int num = 23; // initializing assignment  
num = 45;    // executable assignment
```

- Value can be from **input**, a **literal**, or an **expression**

```
num = readInt("Enter a whole number");  
PI = 3.14159;  
area = PI * radius * radius;
```

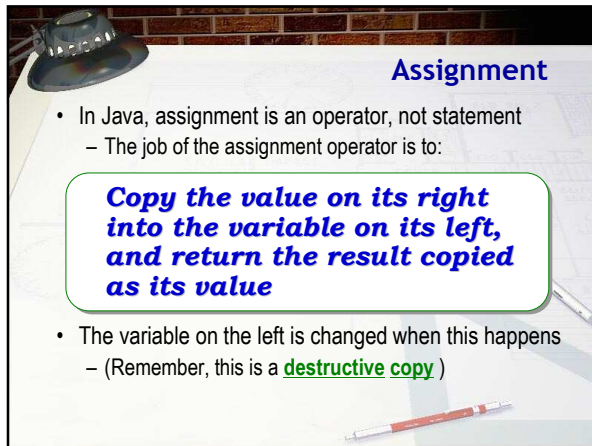
How Assignment Works

- Unlike a mailbox, a variable can hold **exactly one** value
 - Placing a value in a variable, (assignment), **replaces** the value previously stored there

```
myAge = 58;  
kathysAge = myAge;  
kathysAge = 57;
```

- **kathysAge** contains 58, **then** 57
- **myAge** contains 58 for the length of the program

- We say that assignment performs a **destructive copy**

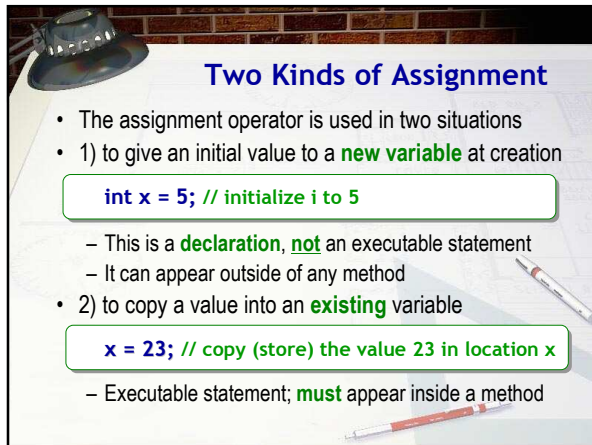


Assignment

- In Java, assignment is an operator, not statement
 - The job of the assignment operator is to:

Copy the value on its right into the variable on its left, and return the result copied as its value

- The variable on the left is changed when this happens
 - (Remember, this is a **destructive copy**)



Two Kinds of Assignment

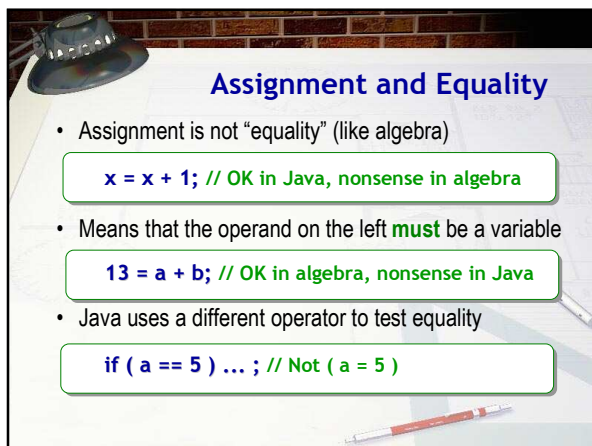
- The assignment operator is used in two situations
- 1) to give an initial value to a **new variable** at creation

```
int x = 5; // initialize i to 5
```

- This is a **declaration**, **not** an executable statement
- It can appear outside of any method
- 2) to copy a value into an **existing** variable

```
x = 23; // copy (store) the value 23 in location x
```

- Executable statement; **must** appear inside a method



Assignment and Equality

- Assignment is not "equality" (like algebra)

```
x = x + 1; // OK in Java, nonsense in algebra
```

- Means that the operand on the left **must** be a variable

```
13 = a + b; // OK in algebra, nonsense in Java
```

- Java uses a different operator to test equality

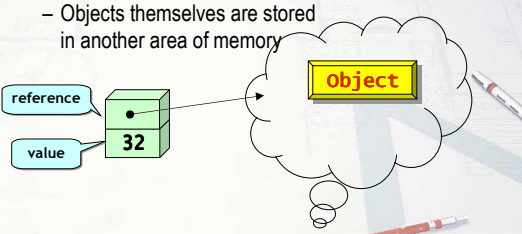
```
if ( a == 5 ) ... ; // Not ( a = 5 )
```

Express Yourself

- **Exercise 12:** Assign values to each of your variables and print them to the console. Shoot a screen-shot
- **Exercise 13:** Assign your **age** variable to your **gpa** and print the result. Shoot a screen-shot
- **Exercise 14:** Assign your **gpa** variable to your **age** variable and print the result. Shoot a screen-shot
- **Exercise 15:** Assign your **age** variable to your **name** variable and print the result. Shoot a screen-shot
- **Exercise 16:** Create another variable named **age**, and assign it a different value. Shoot a screenshot

Object Variables

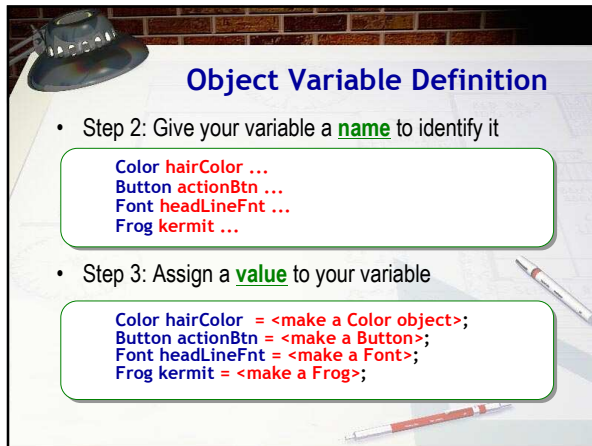
- Types like numbers, are **values** stored in memory
- Object variables, contain **references** (or addresses)
 - Objects themselves are stored in another area of memory



Creating Object Variables

- **Almost** the same as creating a simple variable
 - For every object variable you describe
 - What specific **kind** of thing the variable points to
 - What **name** you want to use for the variable
 - What **object** the variable should refer to
- Step 1: Start with the "**kind**" of thing

Color ...
Button ...
Font ...
Frog ...



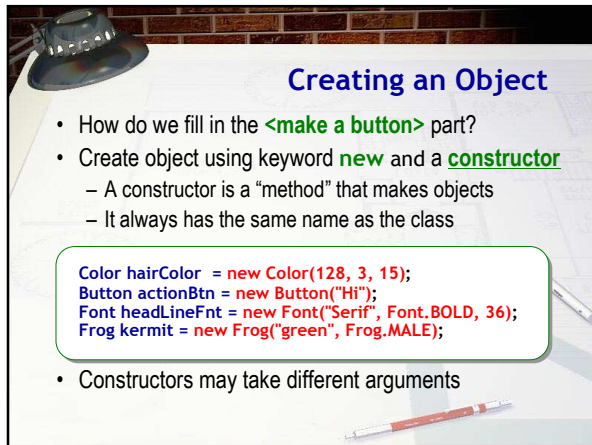
Object Variable Definition

- Step 2: Give your variable a **name** to identify it

```
Color hairColor ...  
Button actionBtn ...  
Font headLineFnt ...  
Frog kermit ...
```

- Step 3: Assign a **value** to your variable

```
Color hairColor = <make a Color object>;  
Button actionBtn = <make a Button>;  
Font headLineFnt = <make a Font>;  
Frog kermit = <make a Frog>;
```

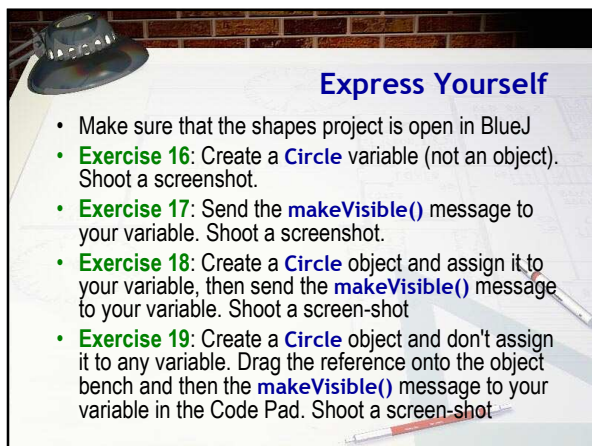


Creating an Object

- How do we fill in the **<make a button>** part?
- Create object using keyword **new** and a **constructor**
 - A constructor is a "method" that makes objects
 - It always has the same name as the class

```
Color hairColor = new Color(128, 3, 15);  
Button actionBtn = new Button("Hi");  
Font headLineFnt = new Font("Serif", Font.BOLD, 36);  
Frog kermit = new Frog("green", Frog.MALE);
```

- Constructors may take different arguments



Express Yourself

- Make sure that the shapes project is open in BlueJ
- Exercise 16:** Create a **Circle** variable (not an object). Shoot a screenshot.
- Exercise 17:** Send the **makeVisible()** message to your variable. Shoot a screenshot.
- Exercise 18:** Create a **Circle** object and assign it to your variable, then send the **makeVisible()** message to your variable. Shoot a screen-shot
- Exercise 19:** Create a **Circle** object and don't assign it to any variable. Drag the reference onto the object bench and then the **makeVisible()** message to your variable in the Code Pad. Shoot a screen-shot

Introducing Karel J. Robot

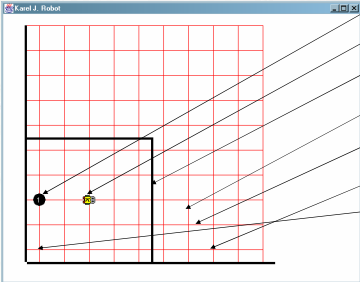
- A suite of Java classes, with a graphical interface, used to illustrate OOP concepts
- We will learn how to write instruction-sets (programs) so that our programmable robots can perform the tasks we give them



World Description

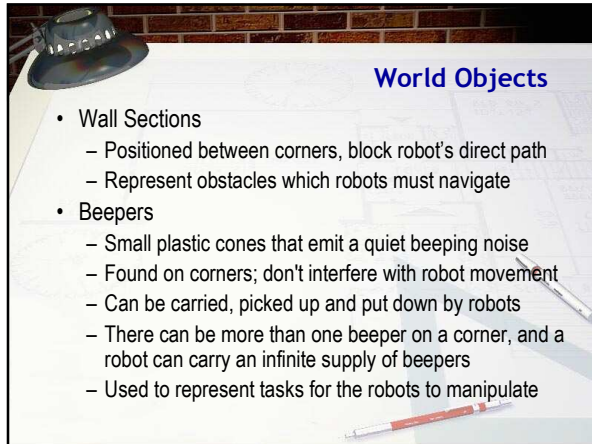
- A great, flat, plane with standard compass points
 - Bounded on west by infinite wall extending north
 - Bounded on south by infinite wall extending east
- Filled with horizontal streets and vertical avenues
 - A corner is an intersection of a street and avenue
 - Robots can occupy any corner, facing any direction
 - Both streets and avenues have numbers
 - Origin is where 1st St and 1st Ave intersect
 - Robots do not have to start from origin

Robot World



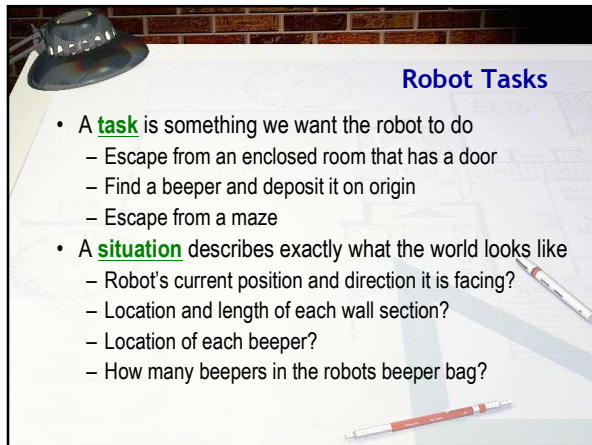
A diagram of the Robot World environment. It shows a grid of streets and avenues. A robot is positioned at a corner. A beeper is also shown. A wall is indicated on the west side. The origin is marked at the intersection of 1st St. and 1st Ave. A north arrow points upwards.

- Beeper
- Robot
- Wall
- Avenue
- Street
- Corner
- Origin
- 1st St., 1st Ave.



World Objects

- Wall Sections
 - Positioned between corners, block robot's direct path
 - Represent obstacles which robots must navigate
- Beepers
 - Small plastic cones that emit a quiet beeping noise
 - Found on corners; don't interfere with robot movement
 - Can be carried, picked up and put down by robots
 - There can be more than one beeper on a corner, and a robot can carry an infinite supply of beepers
 - Used to represent tasks for the robots to manipulate



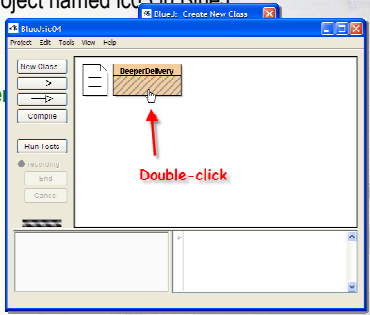
Robot Tasks

- A **task** is something we want the robot to do
 - Escape from an enclosed room that has a door
 - Find a beeper and deposit it on origin
 - Escape from a maze
- A **situation** describes exactly what the world looks like
 - Robot's current position and direction it is facing?
 - Location and length of each wall section?
 - Location of each beeper?
 - How many beepers in the robots beeper bag?



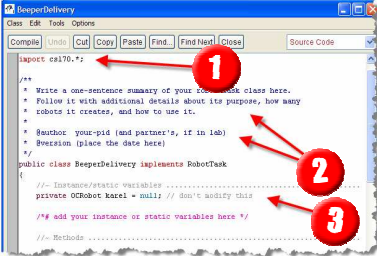
Karel in BlueJ

- Create a new project named ic03 in BlueJ
- Click the **New Class** button
 - BeeperDelivery
 - Robot task class
 - OK
- Open in editor



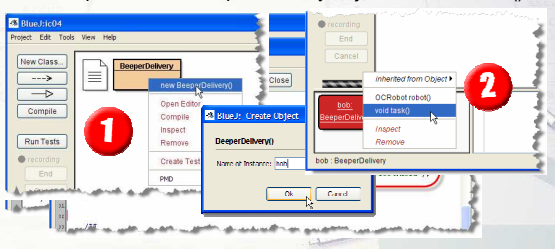
Your First Robot Task

- Notice the code that is "pre-written" for you
 - cs170 package containing **OCRobot** definition
 - Add your name and date, and a description
- Exercise 20:** Shoot a picture
- Note the **OCRobot** variable



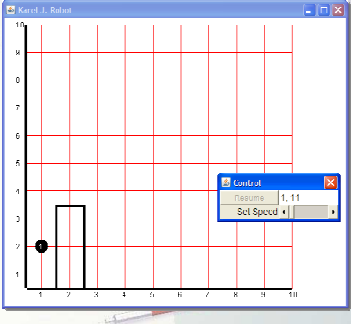
Starting the World

- Locate the `startFromURL()` method and replace with:
 - "http://csjava.occ.cccd.edu/~gilberts/worlds/ic03.kwld"
- Compile, create BeeperDelivery object, and call task()



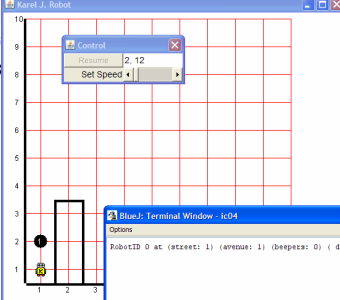
The BeeperDelivery World

- Exercise 21:** Shoot a picture



Adding a Robot

- The world has no robots yet. Locate the magenta line that says: `/*# add`
 - Use the **OCRobot**
 - Compile your task
- Exercise 22:** Shoot a picture of each window that appears



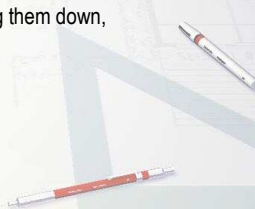
Robot Behavior

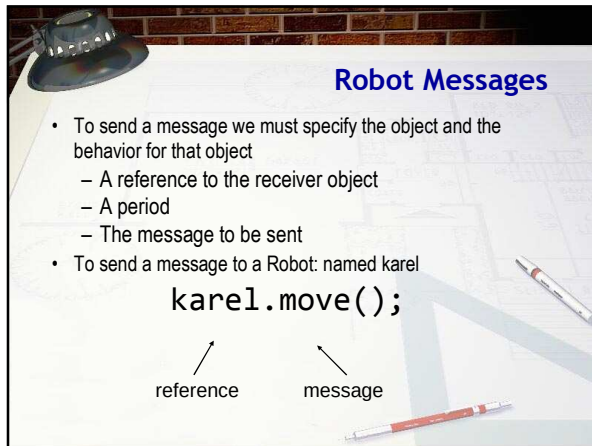
- Note that karel just stands there. What can he do?
- Robots can **move**
 - Forward in direction it is facing, from corner to corner
 - Turn in place
 - Turn itself off
- Robots can **detect**
 - Walls $\frac{1}{2}$ block in front of them
 - Beeper by being on the same corner as a beeper
 - Robots on the same corner with it



More Robot Behavior

- Robots can **navigate** by detecting the direction it is facing (north, south, east, west)
- Robots can **manipulate** beepers
 - by carrying them,
 - picking them up, and putting them down,
 - knowing if it is carrying any



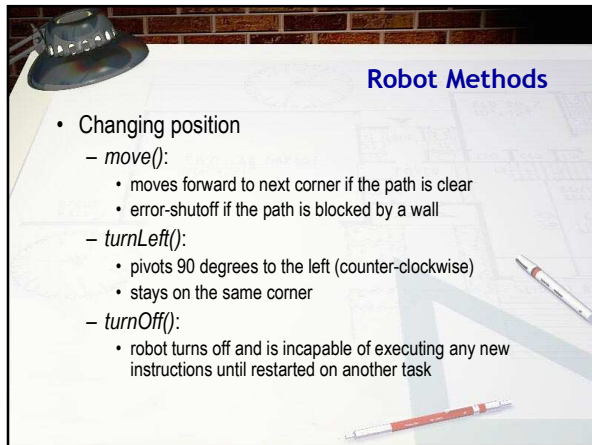


Robot Messages

- To send a message we must specify the object and the behavior for that object
 - A reference to the receiver object
 - A period
 - The message to be sent
- To send a message to a Robot: named karel

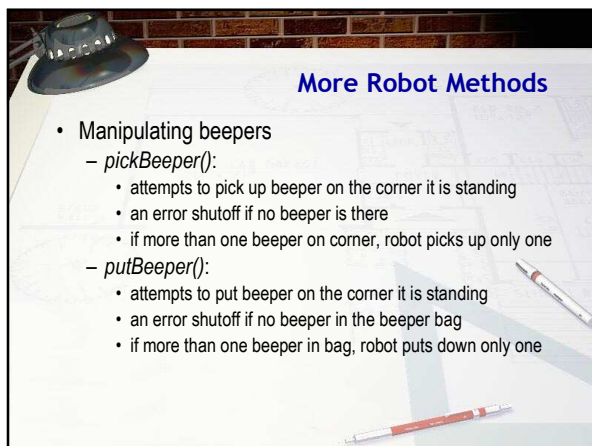
```
karel.move();
```

reference message



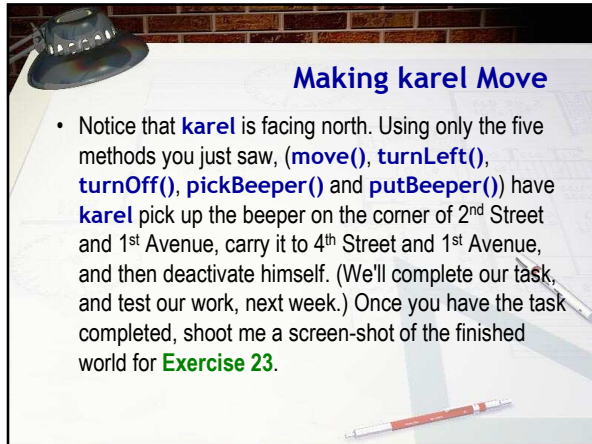
Robot Methods

- Changing position
 - *move()*:
 - moves forward to next corner if the path is clear
 - error-shutoff if the path is blocked by a wall
 - *turnLeft()*:
 - pivots 90 degrees to the left (counter-clockwise)
 - stays on the same corner
 - *turnOff()*:
 - robot turns off and is incapable of executing any new instructions until restarted on another task



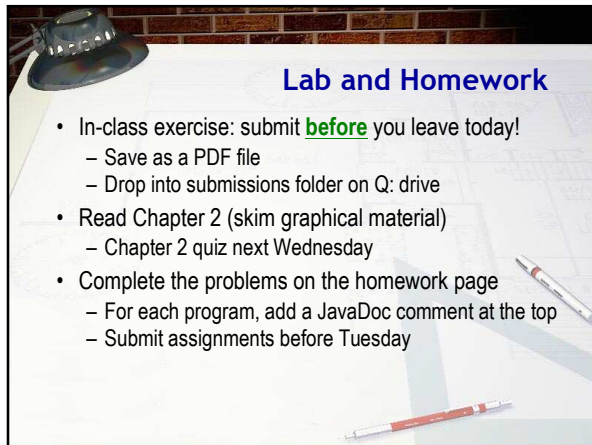
More Robot Methods

- Manipulating beepers
 - *pickBeeper()*:
 - attempts to pick up beeper on the corner it is standing
 - an error shutoff if no beeper is there
 - if more than one beeper on corner, robot picks up only one
 - *putBeeper()*:
 - attempts to put beeper on the corner it is standing
 - an error shutoff if no beeper in the beeper bag
 - if more than one beeper in bag, robot puts down only one



Making karel Move

- Notice that **karel** is facing north. Using only the five methods you just saw, (**move()**, **turnLeft()**, **turnOff()**, **pickBeeper()** and **putBeeper()**) have **karel** pick up the beeper on the corner of 2nd Street and 1st Avenue, carry it to 4th Street and 1st Avenue, and then deactivate himself. (We'll complete our task, and test our work, next week.) Once you have the task completed, shoot me a screen-shot of the finished world for **Exercise 23**.



Lab and Homework

- In-class exercise: submit **before** you leave today!
 - Save as a PDF file
 - Drop into submissions folder on Q: drive
- Read Chapter 2 (skim graphical material)
 - Chapter 2 quiz next Wednesday
- Complete the problems on the homework page
 - For each program, add a JavaDoc comment at the top
 - Submit assignments before Tuesday
